

OPERATING SYSTEMS AND COMPUTER NETWORKS

Fundamentals of Operating Systems & The Windows Operating System

Week 1 • Lecture (2 academic hours)

Today's Agenda

Part 1

What Is an Operating System?

Definition, role, and why every computer needs one

Part 2

Core Functions of an OS

Process, memory, file, and device management

Part 3

Types & Architectures of Operating Systems

Batch, time-sharing, real-time, distributed, mobile; kernel designs

Part 4

The Windows Operating System

History, architecture, configuration & monitoring

Part 5

Windows Security

Accounts, permissions, Defender, updates, BitLocker

Part 6

Wrap-up & Bridge to Practice

Summary, key terms, and this week's lab preview

By the End of This Lecture, You Will Be Able To

- Define what an operating system is and explain why computers need one
- Identify and describe the four core functions of an OS: process, memory, file, and device management
- Distinguish between major OS types (batch, time-sharing, real-time, distributed, mobile) and kernel architectures
- Trace the evolution of Microsoft Windows and describe its layered architecture
- Use core Windows configuration and monitoring tools (Task Manager, Registry, Services, Event Viewer)
- Explain the main Windows security mechanisms and why they matter for everyday use

What Is an Operating System?

An Operating System (OS) is system software that manages computer hardware and software resources, and provides common services for application programs.

Think of the OS as a resource manager and a translator: it sits between raw hardware and the applications you use, hiding hardware complexity from the user.

Applications (browser, editor, games)

Operating System (kernel + services)

Hardware (CPU, RAM, disk, devices)

Why Every Computer Needs an OS

1

Abstraction

Hides the messy details of hardware (disk sectors, memory addresses, device registers) behind simple concepts: files, windows, programs.

2

Resource sharing

Multiple programs must share one CPU, limited RAM, and I/O devices safely and fairly.

3

Convenience & control

Provides a consistent way for users and programs to interact with the machine, and enforces rules so one program cannot crash the whole system.

Four Core Functions of an Operating System

Process Management

Creates, schedules, and terminates programs in execution (processes and threads).

Memory Management

Allocates and tracks RAM among running processes; enables virtual memory.

File System Management

Organizes, stores, retrieves, and protects data as files and folders.

Device Management

Controls I/O devices (disks, printers, network cards) via drivers.

Process Management

- A process is a program in execution — code, data, and its current state (registers, program counter, stack).
- A thread is a lightweight unit of execution within a process; one process can have many threads sharing the same memory.
- The OS scheduler decides which process/thread runs next on the CPU, switching rapidly (context switching) to create the illusion of parallelism.
- Process states: New → Ready → Running → Waiting → Terminated.

Windows Task Manager lets you inspect and manage running processes and threads directly.

Memory Management

- The OS allocates RAM to each running process and reclaims it when the process ends
- Virtual memory lets the system run programs larger than physical RAM by using disk space (a page/swap file) as an extension of memory
- Memory protection keeps one process from reading or corrupting another process's memory — critical for stability and security
- Paging divides memory into fixed-size blocks (pages) that can be moved between RAM and disk as needed

File System & Device Management

File System Management

- Organizes data into files and directories (folders)
- Tracks where each file's data is physically stored on disk
- Provides permissions so users/programs access only what they're allowed to
- Common systems: NTFS, exFAT (Windows); ext4 (Linux)

Device Management

- Manages communication with I/O devices: keyboard, mouse, disk, printer, network adapter
- Uses device drivers — small programs that translate OS commands into device-specific instructions
- Handles interrupts: signals from hardware that need immediate OS attention
- Ensures devices are shared fairly and safely among processes

User Interfaces: CLI vs. GUI

Command-Line Interface (CLI)

- Text-based commands (e.g., cmd, PowerShell, Bash)
- Precise, scriptable, fast for repetitive tasks
- Steeper learning curve for beginners

Graphical User Interface (GUI)

- Windows, icons, menus, pointers
- Intuitive and visual — the default for most end users
- Windows Desktop, File Explorer are GUI examples

Types of Operating Systems

Batch OS — Jobs collected and run in groups with no direct user interaction (early mainframes).

Time-Sharing / Multiprogramming — CPU time is divided among multiple users/programs, giving each the illusion of a dedicated system.

Real-Time OS (RTOS) — Guarantees response within strict time limits — used in medical devices, industrial control.

Distributed OS — Manages a group of networked computers as if they were a single system.

Mobile OS — Optimized for touch, battery life, and connectivity — e.g., Android, iOS.

OS Architecture: Kernel Designs

Monolithic Kernel

All core services (process, memory, file, drivers) run together in one large kernel space. Fast, but a bug in one part can crash the system. Example: Linux.

Microkernel

Only the bare minimum (communication, basic scheduling) runs in kernel space; other services run as separate processes. More stable, some performance cost.

Hybrid Kernel

Combines both approaches for balance of speed and modularity. Example: Windows NT kernel.

PART 4

The Windows Operating System

History • Architecture • Configuration & Monitoring • Security

A Brief History of Windows

Windows 1.0 — a GUI
shell on top of MS-
DOS

1985

Windows XP — merges
consumer/business lines,
long-lived stability

2001

Windows 10 — Windows
as a service, regular
feature updates

2015

1995

Windows 95 — Start
menu, taskbar, plug-
and-play

2009

Windows 7 — refined
performance and UI,
widely adopted

2021+

Windows 11 — modern
UI, tighter hardware
security requirements

Windows Architecture: User Mode & Kernel Mode

User Applications

Word, browsers, games — run with limited privileges

Subsystem DLLs (Win32 API)

Translate app requests into system calls

— User Mode / Kernel Mode boundary —

Executive & Kernel

Process/memory/IO managers, scheduler — core Windows services

Hardware Abstraction Layer (HAL)

Isolates the kernel from hardware differences across devices

Hardware (CPU, RAM, disks, peripherals)

Windows Configuration & Monitoring Tools

Task Manager

View/manage running processes, threads, CPU, memory, and startup programs (Ctrl+Shift+Esc).

Windows Registry (regedit)

Hierarchical database storing OS and application configuration settings.

Services (services.msc)

Manage background processes that start with Windows and run without a user interface.

Event Viewer

Logs of system, security, and application events — the first place to check when troubleshooting.

Control Panel / Settings

Central hub for hardware, network, user account, and system configuration.

Device Manager

View and manage installed hardware devices and their drivers.

Core Windows Security Mechanisms

- User Accounts & Permissions — Administrator vs. Standard user; the principle of least privilege limits what each account can change.
- User Account Control (UAC) — Prompts for confirmation before actions that need elevated privileges.
- Windows Defender (Microsoft Defender Antivirus) — Built-in real-time malware protection and firewall.
- Windows Update — Regular patches close security vulnerabilities; disabling updates is a major risk.
- BitLocker — Full-disk encryption protecting data if a device is lost or stolen.
- NTFS permissions — File/folder-level access control (read, write, execute) per user or group.

Key Takeaways

- An OS manages hardware and software resources and provides a stable interface between applications and hardware.
- Its four core functions: process, memory, file, and device management.
- OS types differ by how they schedule work: batch, time-sharing, real-time, distributed, mobile.
- Windows uses a hybrid kernel architecture with a clear user-mode / kernel-mode boundary and a Hardware Abstraction Layer.
- Practical Windows skills — Task Manager, Registry, Services, Event Viewer — are essential for configuration and troubleshooting.
- Security is built into the OS: accounts, UAC, Defender, updates, BitLocker, and NTFS permissions work together.

Key Terms Glossary

Kernel — The core of the OS with full hardware access

Process — A program in execution

Thread — A lightweight unit of execution within a process

Virtual memory — Using disk space to extend available RAM

Driver — Software that lets the OS control a hardware device

HAL — Layer that isolates the kernel from hardware differences

UAC — Windows prompt for privilege elevation

NTFS — Windows' modern file system with permissions support

LOOKING AHEAD

Practice Session & Lab This Week

- Practice session: Exploring Processes, Threads, Handles, and the Windows Registry (Cisco Operating Systems Basics).
- Independent study (SIS): Windows configuration & monitoring, Windows security — 9 hours.
- Next week: Linux Basics — shell, servers/clients, administration, file system, GUI.

Questions?